Cairo University

Faculty of Engineering

Electronics and Electrical Communications Engineering

Assignment 2

ELC 4028 – Artificial Neural Networks

ID	Section	Name
9220399	2	Shahd Gamal Mahmoud Hussein
9220411	2	Salah El-Din Hassan Hassan Mohamed
9220984	4	Yousef Khaled Omar Mahmoud

Supervised by:

Dr. Mohsen Rashwan
Dr. Mohamed Motaz

Contents

Problem 1 **3**

1 Feature Extraction and MLP Benchmark **3**
1.1 Dataset and Preprocessing . . . 3
1.2 Feature Construction . . . 3
1.2.1 Discrete Cosine Transform Features . . . 3
1.2.2 Principal Component Analysis Features . . . 3
1.2.3 Autoencoder Features . . . 3
1.3 Classifier Design . . . 3
1.4 Integration with the Assignment 1 Benchmark . . . 4
1.5 Summary . . . 4

Problem 2 **5**

2 CNN Experiments (LeNet-style) **5**
2.1 Design Decisions . . . 5
2.2 Mathematical and Implementation Details . . . 5
2.3 Experimental Protocol . . . 5
2.4 Results and Findings . . . 5
2.5 Discussion . . . 6
2.6 Conclusion and Recommendations . . . 6

Problem 3 **7**

3 Placeholder **7**
3.1 Results . . . 7

Problem 4 **8**

4 Placeholder **8**
4.1 Results . . . 8

Problem 5 **9**

5 Placeholder **9**
5.1 Results . . . 9

Problem 6 **10**

6 Placeholder **10**
6.1 Results . . . 10

Conclusion **11**

List of Figures

1 Per-epoch train/test accuracy curves for LeNet variants. 6

Problem 1

1 Feature Extraction and MLP Benchmark

1.1 Dataset and Preprocessing

The Reduced MNIST dataset was downloaded through `kagglehub` and already provided in labeled training and testing folders, one subdirectory per digit class. In our implementation, each image was loaded from disk, converted again to grayscale to ensure a single-channel input, and normalized to the range $[0, 1]$ by dividing pixel intensities by 255.

The label of each sample was taken directly from the folder name. No relabeling or manual reshuffling was introduced, which keeps the preprocessing aligned with the assignment dataset structure.

1.2 Feature Construction

The assignment required three feature representations: DCT, PCA, and autoencoder features. We implemented each feature extractor separately so that the same classifier could be evaluated on all representations under a fair comparison.

1.2.1 Discrete Cosine Transform Features

For the DCT representation, each grayscale image was transformed using a two-dimensional discrete cosine transform with `scipy.fft.dctn`. We used the orthonormal DCT-II variant and retained the top-left 15×15 low-frequency block, which captures the most informative spatial structure while reducing the original 28×28 image to a compact 225-dimensional vector.

1.2.2 Principal Component Analysis Features

For PCA, each image was first flattened into a 784-dimensional vector and then projected onto the principal subspace using `sklearn.decomposition.PCA`. We configured PCA with `n_components=0.95`, which preserves 95% of the variance in the training data.

1.2.3 Autoencoder Features

We implemented it as part of the same feature pipeline so that its learned latent space could be compared directly with DCT and PCA.

The autoencoder is implemented as a multilayer perceptron regressor with symmetric hidden layers `(128, 64, 32, 64, 128)`. It is trained to reconstruct the input image using ReLU activations and early stopping. The bottleneck is the middle hidden layer with 32 units. After training, we pass the data forward layer by layer and extract the bottleneck activations as the learned feature vector for each image.

1.3 Classifier Design

After extracting features, we trained a multilayer perceptron classifier on each feature set. The classifier uses ReLU activations, Adam optimization, early stopping, and a maximum of 120 iterations.

We evaluated three hidden-layer configurations:

- one hidden layer: `(512,)`
- three hidden layers: `(512, 256, 128)`
- four hidden layers: `(512, 256, 128, 64)`

These configurations were chosen to study the effect of model depth on classification performance while keeping the feature extractor fixed.

1.4 Integration with the Assignment 1 Benchmark

The autoencoder implementation was also reused later in Problem 2 because the assignment table required us to rerun the Assignment 1 Part 2 benchmark using this feature.

1.5 Summary

In summary, Problem 1 focuses on the feature-extraction stage of the experiment. The three required features were implemented exactly as separate pipelines, and the same MLP classifier was applied to all of them using three hidden-layer configurations. The detailed accuracy and timing results are documented in the results table of Problem 2.

Problem 2

2 CNN Experiments (LeNet-style)

2.1 Design Decisions

We implemented a baseline `LeNet5` with the following structure:

- Convolutional layer 1: 5×5, 6 filters, followed by average pooling (2x2)
- Convolutional layer 2: 5×5, 16 filters, followed by average pooling (2x2)
- Fully connected layers: flatten $\rightarrow$ 120 $\rightarrow$ 84 $\rightarrow$ 10 (logits)

Variants added to investigate capacity and regularisation effects:

Wide: increase conv filters to (16,32) to raise model capacity.

Deep: add an extra 3×3 conv layer (padding=1) increasing depth and receptive field.

Sigmoid: replace ReLU activations in the convolutional and FC layers with Sigmoid activations (no dropout).

Each variant is constructed by a factory function and run with the same training pipeline to ensure fair comparison.

2.2 Mathematical and Implementation Details

Convolutional operations are denoted as

$$y_k = f\left(\sum_c x_c * w_{c,k} + b_k\right),$$

where $*$ is 2D convolution and $f(\cdot)$ is the activation function (ReLU or Sigmoid for the `sigmoid` variant).

Training minimises cross-entropy loss over the dataset. We measure per-epoch wall-clock training time t_{epoch} and report total training time $T = \sum_e t_{\text{epoch}}^{(e)}$.

2.3 Experimental Protocol

- Data preprocessing: grayscale conversion and tensor transform (placeholders for normalization values).
- Optimizer: Adam with learning rate 0.001 (default; configurable).
- Batch size: 128; Epochs: 5 (default; configurable).
- Metrics: train accuracy, test accuracy, training wall time, testing wall time.

2.4 Results and Findings

The comparative results for the four variants are saved to a CSV (`lenet_variants_results.csv`) and should be inserted as a table here. In addition, the table-driven requirement from Assignment 1 Part 2 was revisited using the autoencoder feature, so those outputs are also included in the report workflow for direct comparison. Example placeholder table:

Table 1: LeNet variants: accuracy and timing summary

Variant	Train Acc (%)	Test Acc (%)	Train Time (s)	Test Time (s)
Baseline	–	–	–	–
Wide	–	–	–	–
Deep	–	–	–	–
Sigmoid	–	–	–	–

Placeholder: prob2_accuracy_curves.png

Figure 1: Per-epoch train/test accuracy curves for LeNet variants.

2.5 Discussion

Discuss why the wide or deep variants behaved differently from baseline, how Sigmoid activation affected training dynamics, and any regularisation or optimisation observations (e.g., learning rate sensitivity).

2.6 Conclusion and Recommendations

Summarise best-performing variant, trade-offs between accuracy and compute (training time), and recommendations for further experiments (longer training, data augmentation, normalization).

Problem 3

3 Placeholder

3.1 Results

Place figures and tables for Problem 3 in this file.

Problem 4

4 Placeholder

4.1 Results

Place figures and tables for Problem 4 in this file.

Problem 5

5 Placeholder

5.1 Results

Place figures and tables for Problem 5 in this file.

Problem 6

6 Placeholder

6.1 Results

Place figures and tables for Problem 6 in this file.

Conclusion

Summarize key findings across all problems, reflect on the learning outcomes, and suggest potential future directions or improvements for the experiments conducted.